

Institución:

Universidad de Costa Rica

Nombre

Steve Puerto Cordero

Carné

C4I552

Tarea Corta #04

Docente

Sleyter Angulo

Fecha de la entrega

1 de oct. de 26

INTRODUCCIÓN.

La tarea 04 consiste en la inclusión de semáforos al código del server que ha sido previamente compartido. Esta inclusión de semáforos se da sobre el código usado y desarrollado en la tarea03. Los semáforos son una variable de tipo especial que gestiona/procesa el acceso de los hilos a un recurso compartido, esto con el fin de evitar errores como las condiciones de carrera. Los semáforos actúan como “condiciones” que almacenan información de cada “slot” o espacios N en la memoria.

SOLUCIÓN

La solución es agregar dos variables de tipo sem_t. “Sem_t” es una variable de tipo semáforo que permite manejar slots/espacios para almacenar esta información. Para agregar los semáforos, el “sem_t empty” almacenará los espacios que tenemos disponibles en la cola, y el “sem_t full” manejará los espacios que ya han sido rellenos. Estos elementos son inicializados con:

sem_init (&qp->empty, 0, N); el 0 muestra que es compartido entre hilos del mismo proceso, y tiene N (valor inicial) espacios disponibles.

sem_init (&qp->full, 0, 0); el 0 muestra que es compartido entre hilos del mismo proceso, y tiene el valor inicial es 0.

Cuando el productor quiera añadir un elemento a la cola, primero le consulta a &empty mediante el sem_wait(), si retorna 0, espera a que un espacio sea liberado. Proceso inverso para el consumidor, que primero consulta el &full y si retorna 0, debe esperar. El sem_post(&empty) y sem_post(&full) añaden 1; en el full, hay un elemento nuevo que debe ser consumido, en empty que hay una casilla menos que puede ser usada. Lo que cambió respecto a la solución de la tarea 03, es que se usó condiciones, y para este caso, se usa el semáforo. Además, se elimina el int “counter” que existía en los atributos del struct queue_t, debido a que los semáforos de este tipo tienen automáticamente un contador.

ANÁLISIS DE RESULTADOS

1 CORE.

Corrida 1	Corrida 2	Corrida 3	Core(s)	Hilos	Paquetes
28.511 ms	31.374 ms	29.210 ms	1	1	150
20.438 ms	24.044 ms	15.340 ms	1	2	150
31.875 ms	24.124 ms	24.725 ms	1	4	150
20.575 ms	33.733 ms	20.698 ms	1	8	150

Tabla 1. Recolección de los tiempos generados por el server-client con un core.

```
● user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 28.511 ms
● user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 31.374 ms
● user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 29.210 ms
● user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 20.438 ms
● user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 24.044 ms
● user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 15.340 ms
● user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 31.875 ms
● user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 24.124 ms
● user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 24.725 ms
● user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 20.575 ms
● user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 33.733 ms
● user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 20.698 ms
○ user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
```

Figura 1. Evidencia, tiempos generados por cada corrida; 3 ejecuciones para cada server con un 1 core, diferentes hilos y 150 paquetes.

2 CORES

Corrida 1	Corrida 2	Corrida 3	Core(s)	Hilos	Paquetes
11.470 ms	20.623 ms	10.439 ms	2	1	150
51.228 ms	43.309 ms	50.797 ms	2	2	300
41.302 ms	50.645 ms	51.475 ms	2	4	300

Tabla 2. Recolección de tiempos generados por el server-client con dos cores.

```
user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ gcc load_client.c net_util.c -o load_client
user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 11.470 ms
user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 20.623 ms
user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 15
requests completed: 150
needed time: 10.439 ms
```

Figura 2. Tiempos generados por 2 cores con 1 hilo y procesando 150 paquetes.

```
user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ gcc load_client.c net_util.c -o load_client
user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 30
requests completed: 300
needed time: 51.228 ms
user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 30
requests completed: 300
needed time: 43.309 ms
user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 30
requests completed: 300
needed time: 50.797 ms
user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$
```

Figura 3. Tiempos generados por 2 cores, con 2 hilos y procesando 300 paquetes.

```
user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ gcc load_client.c net_util.c -o load_client
user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 30
requests completed: 300
needed time: 41.302 ms
user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 30
requests completed: 300
needed time: 50.645 ms
user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$ ./load_client 127.0.0.1 8080 10 30
requests completed: 300
needed time: 51.475 ms
user@DESKTOP-39508PN:/mnt/c/Users/User/Desktop/trabajo04/tarea04/src$
```

Figura 4. Tiempos generados por 2 cores, con 4 hilos y procesando 300 paquetes.

Pasar de 1 a 2 cores con 1 hilo redujo el promedio de 29.698 a 14.177 ms (una reducción del 52%, o 2.1 veces más paquetes por segundo). Aun así, cuando se tienen 2 cores tiene el CV más alto (39.5), porque una corrida (20.6 ms) se aleja bastante de las otras dos (10.4 y 11.5 ms). Además, como el cliente corre en la misma máquina, parte de la mejora puede deberse a que cliente y servidor dejan de competir por el mismo core, y no solo al servidor. Los semáforos no generaron bloqueos ni pérdidas en ninguna de las 21 corridas. Y vemos que la mejora en las métricas se genera principalmente por la cantidad de cores. El semáforo ofrece seguridad en términos del manejo de los datos, los cores en términos de tiempo. El semáforo hace que el hilo se bloquee y duerma mientras no hay espacio o elementos, en lugar de consultar la cola una y otra vez, y el contador de espacios lo lleva el propio semáforo. Es un uso eficiente, un uso donde se puede controlar lo que se tiene disponible, evitando así el intento de acceso de recursos que no se tienen.

CONCLUSIONES

Con la implementación de los semáforos full y empty, el server logró atender todas las peticiones en cada una de las corridas. En todos los casos propuestos se completó sin que el programa se quedara bloqueado ni perdiera conexiones. Esto confirma que los semáforos sí funcionaron de la manera esperada, ya que el productor espera cuando la cola está llena y los consumidores esperan cuando está vacía. Además, se mantiene el mutex para proteger los índices `index_in` e `index_out`, debido a que los semáforos controlan cuántos espacios hay, pero no evitan que dos hilos modifiquen el mismo índice al mismo tiempo

En cuanto a los tiempos, con 1 core no se observó una mejora significativa al aumentar la cantidad de hilos. Los promedios fueron de 29.698 ms con 1 hilo, 19.941 ms con 2 hilos, 26.908 ms con 4 hilos y 25.002 ms con 8 hilos, la diferencia entre ellos puede ser relacionada al sistema operativo ejecutado en la computadora donde se realizaron las pruebas. Esto tiene sentido ya que con un solo core los hilos no corren en paralelo, solo se turnan, y el trabajo de cada conexión es muy corto.

Al pasar de 1 a 2 cores con 1 hilo y 150 paquetes, el promedio bajó de 29.698 ms a 14.177 ms, es decir, casi la mitad (ver tabla 1 y 2, la figura 1 y 2). En los escenarios de 300 paquetes con 2 cores, el tiempo fue de 48.445 ms con 2 hilos y de 47.807 ms con 4 hilos, prácticamente iguales. Además, estos tiempos son más altos que los de 150 paquetes, porque se está atendiendo el doble de conexiones. Por esa

razón, no se puede comparar directamente las filas de 150 paquetes con las de 300 para medir el efecto de los hilos.

Como limitaciones, las pruebas se hicieron en localhost, con solo 3 corridas por escenario y con un tamaño de cola fijo de $N = 32$. Además, el cliente que tenía 10 hilos, corrió en la misma máquina y también consume CPU, y puede haber generado un cambio en los tiempos. En general, se puede concluir que los semáforos resuelven correctamente la sincronización de la cola, y que la cantidad de cores influye más en el tiempo que la cantidad de hilos para este tamaño de carga.

REPOSITORIO

<https://github.com/puertost14-art/tarea04>

REFERENCIAS BIBLIOGRÁFICAS

Microsoft. (2025, 19 de mayo). *Windows Subsystem for Linux documentation*.

Microsoft Learn. <https://learn.microsoft.com/en-us/windows/wsl/>

The Open Group. (2018). *semaphore.h*. En *The Open Group Base Specifications Issue 7, 2018 edition*.

<https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/semaphore.h.html>

Arch Linux. (2026). *semaphore.h(0P)—POSIX Programmer's Manual*. Arch manual pages. <https://man.archlinux.org/man/core/man-pages/semaphore.h.0p.en>